Quest 12: The Castastrophe

Lord of the Wings · covering magma with straight-line spells

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION
The grid holds stacks of magma blocks; one spell clears a straight horizontal or vertical run of touching blocks (it can't jump over a gap). Find the fewest spells that clear every block.

INPUT
A grid of `#` (magma) and `.` (empty).

OUTPUT
The minimum total number of spells.

WHAT CHANGES BETWEEN PARTS
The rule never changes, only the shapes. Part I: every stack is a rectangle. Part II: rectangles and L-shapes (any orientation and arm thickness). Part III: rectangles with a rectangular hole cut out, which can split rows and columns in two, and other stacks can sit inside a hole.

The story

Just before the last portal, the cursed egg takes control of Frodoff, who shoves the narrator into the Abyss of Unmarked Recursion. Waiting there is Stackrog, an old enemy who throws stacks of magma. The narrator remembers a reflection spell that bounces back one straight line of blocks at a time, and has to save energy by casting as few spells as possible while Stackrog's stacks get more and more awkward.

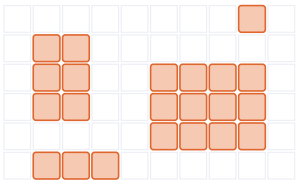
Understanding the question

The rules

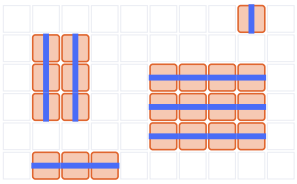
- A **stack** is a group of `#` cells joined up, down, left or right. Stacks never touch, so each can be solved on its own and the answers added up.
- A spell covers a straight run of `#` in one row or one column. It stops at the first `.`, so a row like `###..###` needs two horizontal spells.
- Every block must be covered by at least one spell. Covering a block twice is allowed, but never needed.
- A spell might as well run along the **whole** run it's in: going further never hurts. So the only choices are *which* maximal row-runs and column-runs to cast.

That last point gives the key way to look at it: every block sits on exactly one maximal horizontal run and one maximal vertical run, and **at least one of those two must be cast**.

The Part I example



the notes



7 spells (blue lines)

Extra examples of my own (all checked with my Part III solution)

Stack	Spells	Why
`###` `###`	2	A 3 by 2 rectangle: 2 rows beat 3 columns.
`#####`	1	A straight line is one spell in its own direction.

`###`
`#.#`
`###`
	4	A ring: top row, bottom row, then the two middle blocks need one spell each. Columns also cost 4, because the hole splits the middle column into two runs.

`####`
`####`
`##..`
`##..`
	4	A thick L: rows cost 4, columns cost 4, and 2 columns for the left arm + 2 rows for the top arm is also 4.

Shared approach: reading the grid

All three parts read the map into a grid where each cell is either magma or empty. How the stacks are found and priced is what changes from part to part.

PART I Rectangles only

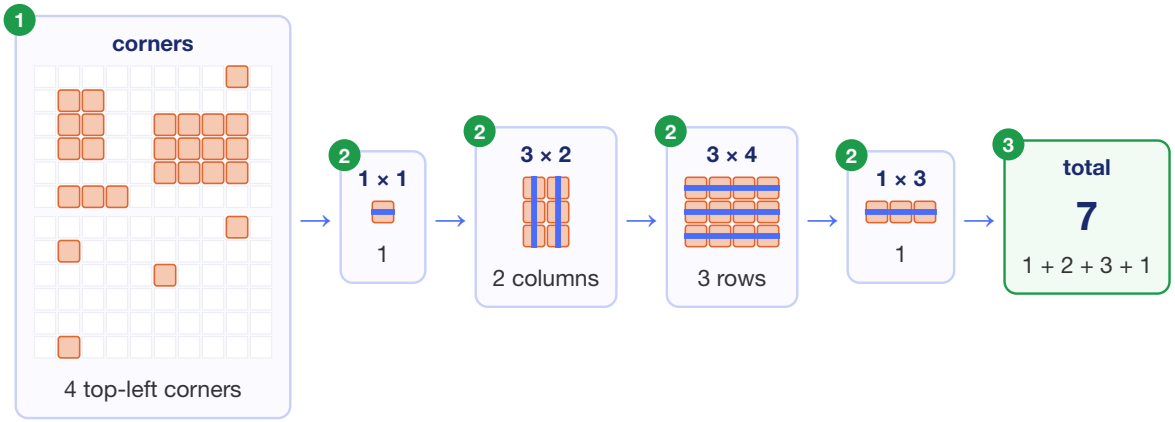
Every stack is a solid rectangle. A rectangle of height *h* and width *w* needs **min(h, w)** spells: one per row or one per column, whichever is fewer. Mixing directions never helps, because any column you don't cast leaves blocks in every row that still need a row spell.

Step-by-step solution

1. Scan the grid for blocks whose upper and left neighbours are empty: these are the top-left corners of the rectangles.
2. From each corner, count blocks going down (height) and going right (width).
3. Add min(height, width) for each rectangle and print the total.

How my solution works

Scan for **top-left corners** (a block with nothing above it and nothing to its left), measure each rectangle by walking down and right from its corner, and add min(height, width):



Dots match the step numbers above. A 5 × 5 square needs 5 either way; a 1 × 100 line needs 1.

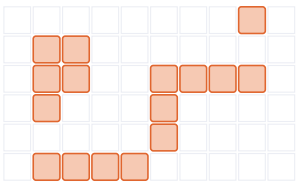
Cost: one pass over the grid, plus a walk down and right from each corner: about O(rows × cols).

Only correct for rectangles (which is all Part I promises). The solution looks at just the top row and left column of each stack. On Part II's first example, a copy of it finds the right-hand L as a 3 × 4 "rectangle" and prints 7 instead of 6; on `sample2.txt` it prints 21 instead of 23. That's why Part II needs a different way of finding stacks.

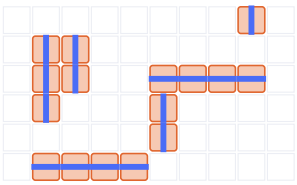
PART II L-shapes

Now some stacks are L-shapes: a rectangle with a rectangular piece missing from one corner. The arms can have any length and thickness, and the missing corner can be any of the four.

Worked examples



first example



6 spells

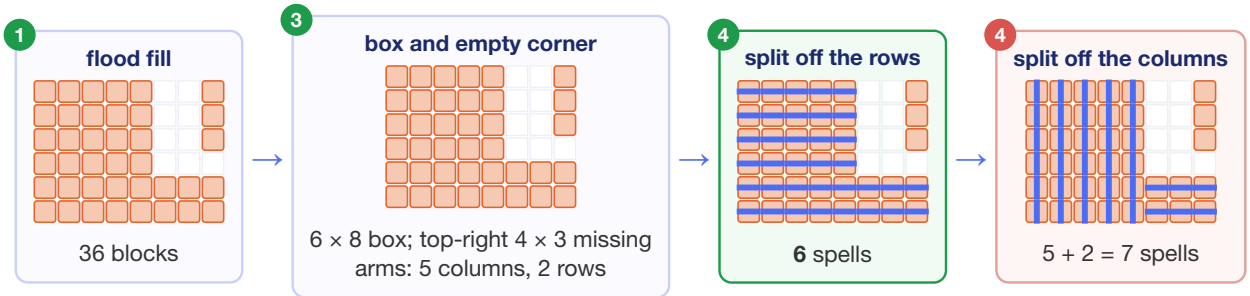
sample2.txt stack	4×3 rectangle	thick L	upper-right L	line	line	block	lower-right L	Total
Spells	3	6	5	1	1	1	6	**23**

Step-by-step solution

1. Scan the grid; at each block not seen yet, flood-fill (BFS) to collect its whole stack.
2. Work out the stack's bounding box. If the stack fills it, it's a rectangle: min(height, width).
3. Otherwise find the empty corner and measure the missing rectangle, then the thickness of the two arms.
4. Try splitting off the full-height arm first or the full-width arm first, covering each piece the cheaper way, and keep the cheaper total.
5. Add up all stacks and print.

How my solution works

Find each stack with a **flood fill**, take its bounding box (a full box is a rectangle: min(H, W)), otherwise find the empty corner, measure the two arms, and try both ways of splitting the L, keeping the cheaper. The thick L from `sample2.txt`:



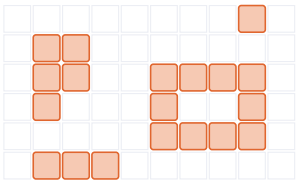
Dots match the step numbers above. In effect the answer is the cheapest of: every row (H), every column (W), or columns through the tall arm plus rows through the wide arm. Flood fill also keeps apart stacks whose boxes overlap but never touch, which Part I's corner trick couldn't. On 2,000 random L-shapes this matched the true minimum (from my Part III solution) every time.

> **Fragile outside Part II's promise:** a non-rectangle is assumed to be an L. With no empty corner (a ring) it crashes; with two (a T-shape) it uses the first and can overcount (3 instead of 2). Neither happens in Part II.

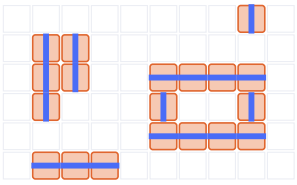
PART III Rectangles with holes

Some stacks are now rectangles with a rectangular hole. A hole can break a row (or column) into two separate runs, each needing its own spell, and another stack can sit inside the hole. Shape-by-shape formulas get messy, so a general method helps.

Worked examples



first example



8 spells: the ring needs 4

Example	Stacks	Horizontal runs	Vertical runs	Answer
first example (above)	4	9	12	8
second example (7 stacks)	7	36	70	29
sample3.txt (nested)		29	93	16

My Part III solution produced all three answers, and also gives 7, 6 and 23 on the Part I and II examples.

The idea: a bipartite graph and König's theorem

Make one node for every maximal **horizontal run** and one for every maximal **vertical run**. Each block lies on exactly one of each, so draw an edge between them for each block. A set of spells covers every block exactly when it touches every edge: that's a **minimum vertex cover**. The graph is **bipartite** (edges only go horizontal ↔ vertical), and **König's theorem** says that in a bipartite graph the minimum vertex cover has the same size as the **maximum matching** (the most edges you can pick with no two sharing a node). A maximum matching can be found with augmenting paths (**Kuhn's algorithm**).

Step-by-step solution

1. Give every maximal horizontal run of blocks an id, and every maximal vertical run an id.
2. For each block, connect its horizontal run to its vertical run (a bipartite graph).
3. Find a maximum matching: for each horizontal run, look for an augmenting path (take a free vertical run, or move the current owner of one to another).
4. Print the number of matched pairs: by König's theorem it equals the fewest spells.

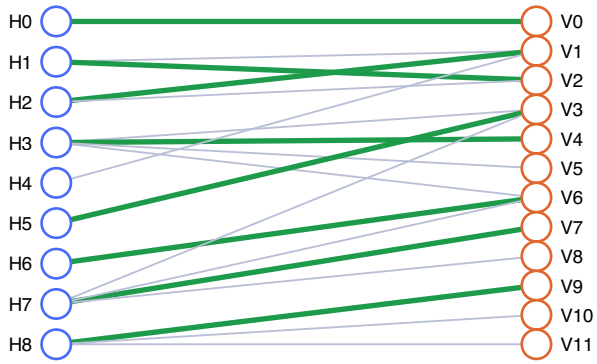
How my solution works

Label every horizontal and vertical run, connect the two runs of each block, and grow a **maximum matching**: for each horizontal run, take a free vertical run it crosses, or free one up by moving its current partner elsewhere (an **augmenting path**). The number of matches is the answer.

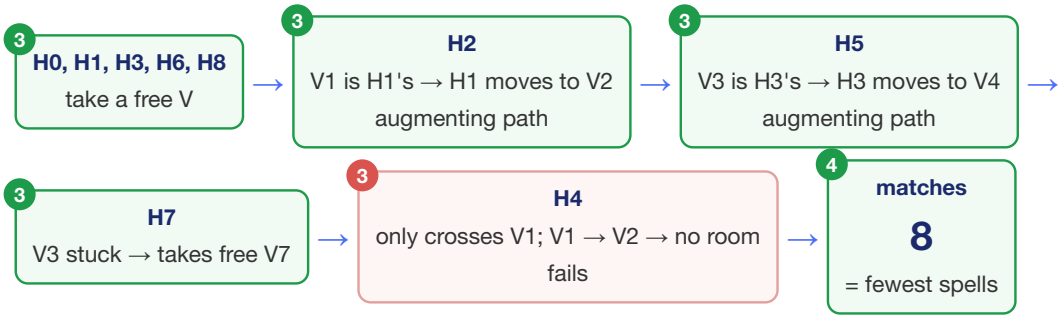
Trace: the first example

Run ids from an instrumented copy (H = horizontal, V = vertical):

```
horizontal run ids                vertical run ids
.  .  .  .  .  .  .  .  0  .      .  .  .  .  .  .  .  .  0  .
.  1  1  .  .  .  .  .  .  .      .  1  2  .  .  .  .  .  .  .
.  2  2  .  .  3  3  3  3  .      .  1  2  .  .  3  4  5  6  .
.  4  .  .  .  5  .  .  6  .      .  1  .  .  .  3  .  .  6  .
.  .  .  .  .  7  7  7  7  .      .  .  .  .  .  3  7  8  6  .
.  8  8  8  .  .  .  .  .  .      .  9 10 11  .  .  .  .  .  .
```



edges = blocks; green = the final matching (8 edges)



Dots match the step numbers above. H4 failing is the left L at work: its three rows only cross its two columns, so two vertical spells beat three horizontal ones.

Cost: each augmenting-path search can visit every edge, so O(H-runs × blocks) at worst; in practice far less. The real input runs in about 0.07 s.
Harmless detail: the solution finds the *size* of the best cover, not which spells to cast; that's all the question asks.

Recursion depth. The augmenting-path search goes one level deeper for every step along the path, so on a very large grid with very long paths it could run out of stack space. Nowhere near that here.

Summary

Part	Shapes	How a stack is found	Spells per stack	Sample answer
I	rectangles	top-left corner, walk right and down	min(H, W)	7
II	+ L-shapes	BFS flood fill + bounding box	min of two arm splits	23
III	+ rectangles with holes, nested	not needed: one global graph	maximum bipartite matching	16

Key ideas: each block forces "its row run or its column run", which makes this a **minimum vertex cover** on a **bipartite graph**. **König's theorem** turns that into a **maximum matching**, found with **augmenting paths** (Kuhn's algorithm). For simple shapes a direct formula works, but it breaks as soon as the shapes change.